Développement Web Avancé

Framework Laravel 11

Chapitre 4 : Bases de Données et Migrations

Enseignant
M. JARRAY Mondher

Classe
MP1ILC

Semestre
S1

Durée
1h30

Objectifs du Chapitre

- ✓ Configurer la connexion à la base de données
- ✓ Comprendre et créer des migrations
- ✓ Définir la structure des tables avec les migrations
- ✓ Utiliser les seeders pour remplir les tables
- ✓ Utiliser les factories pour générer des données de test
- ✓ Concevoir le schéma de la base TaskFlow

Plan du Chapitre

1. **Configuration de la Base de Données**
   - Le fichier .env
   - SQLite vs MySQL
2. **Les Migrations**
   - Qu'est-ce qu'une migration ?
   - Créer et exécuter des migrations
   - Types de colonnes
3. **Clés Étrangères et Relations**
4. **Les Seeders**
5. **Les Factories**
6. **Schéma TaskFlow**

1. Configuration de la Base de Données

Fichier .env

La configuration de la base de données se fait dans le fichier `.env`

Configuration MySQL

```env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=taskflow
DB_USERNAME=root
DB_PASSWORD=
```

Configuration SQLite (Laravel 11 par défaut)

```env
DB_CONNECTION=sqlite
# Le fichier sera créé dans database/database.sqlite
```

SQLite

- Pas de serveur à installer
- Un seul fichier
- Idéal pour le développement

2. Qu'est-ce qu'une Migration ?

Versionnement de la base de données

Définition

Une **migration** est un fichier PHP qui décrit les modifications à apporter à la structure de la base de données. C'est comme un "historique des changements" pour vos tables.

Pourquoi utiliser les migrations ?

❌ Sans migrations

- Scripts SQL manuels
- Difficile à partager
- Pas de versionnement
- Erreurs fréquentes

✅ Avec migrations

- Code PHP versionné
- Partageable via Git
- Annulation possible
- Équipe synchronisée

Créer une Migration

Commande Artisan

Créer une migration pour une nouvelle table

```
$ php artisan make:migration create_projets_table
```

Structure du fichier généré

PHP

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Exécuter la migration (créer la table)
     */
    public function up(): void
    {
        Schema::create('projets', function (Blueprint $table) {
            $table->id();
            $table->timestamps();
        });
    }
```

Types de Colonnes

Définir la structure de vos tables

Méthode	Type SQL	Exemple
`id()`	BIGINT AUTO_INCREMENT	Clé primaire
`string('nom')`	VARCHAR(255)	Texte court
`text('description')`	TEXT	Texte long
`integer('age')`	INTEGER	Nombre entier
`boolean('actif')`	BOOLEAN	Vrai/Faux
`date('date_debut')`	DATE	Date
`datetime('publie_le')`	DATETIME	Date et heure
`timestamps()`	created_at, updated_at	Dates auto
`enum('statut', [...])`	ENUM	Valeurs limitées
`foreignId('user_id')`	BIGINT UNSIGNED	Clé étrangère

Modificateurs de Colonnes

Options supplémentaires

PHP

```php
Schema::create('projets', function (Blueprint $table) {
    $table->id();

    // Obligatoire (par défaut)
    $table->string('nom');

    // Nullable (peut être vide)
    $table->text('description')->nullable();

    // Valeur par défaut
    $table->string('statut')->default('en_attente');

    // Valeur unique
    $table->string('email')->unique();

    // Longueur personnalisée
    $table->string('code', 10);

    // Index pour recherche rapide
    $table->string('slug')->index();

    // Combinaison
    $table->date('date_fin')->nullable()->index();

    $table->timestamps();
});
```

Exemple : Table Projets

Migration complète pour TaskFlow

PHP

```php
public function up(): void
{
    Schema::create('projets', function (Blueprint $table) {
        // Clé primaire
        $table->id();

        // Informations du projet
        $table->string('nom');
        $table->text('description')->nullable();

        // Statut avec valeur par défaut
        $table->enum('statut', ['en_attente', 'en_cours', 'termine'])
              ->default('en_attente');

        // Dates
        $table->date('date_debut')->nullable();
        $table->date('date_fin')->nullable();

        // Propriétaire du projet (clé étrangère)
        $table->foreignId('user_id')
              ->constrained()
              ->onDelete('cascade');

        // Timestamps automatiques
        $table->timestamps();
    });
}
```

Commandes de Migration

Exécuter et gérer les migrations

Commande	Description
`php artisan migrate`	Exécuter toutes les migrations en attente
`php artisan migrate:status`	Voir l'état des migrations
`php artisan migrate:rollback`	Annuler la dernière migration
`php artisan migrate:reset`	Annuler toutes les migrations
`php artisan migrate:fresh`	Supprimer toutes les tables et re-migrer
`php artisan migrate:refresh`	Rollback + migrate

Attention

`migrate:fresh` et `migrate:reset` suppriment toutes les données ! À utiliser uniquement en développement.

Commande courante en développement

```
$ php artisan migrate:fresh --seed
```

Recréer toutes les tables et insérer les données de test.

3. Clés Étrangères

Lier les tables entre elles

Méthode moderne (Laravel 7+)

```php
// Crée une colonne user_id + contrainte de clé étrangère
$table->foreignId('user_id')->constrained();

// Avec action de suppression
$table->foreignId('user_id')
      ->constrained()
      ->onDelete('cascade');  // Supprimer les projets si user supprimé

// Référencer une autre table
$table->foreignId('projet_id')
      ->constrained('projets')
      ->onDelete('cascade');
```

Actions de suppression

Action	Comportement
`cascade`	Supprimer les enregistrements liés
`restrict`	Empêcher la suppression si des liens existent
`set null`	Mettre la clé étrangère à NULL

4. Les Seeders

Remplir la base avec des données

Définition

Un **seeder** est une classe qui insère des données dans la base de données. Utile pour les données de test ou les données initiales.

Créer un seeder

```
$ php artisan make:seeder ProjetSeeder
```

Exemple de seeder

PHP

```php
// database/seeders/ProjetSeeder.php
use App\Models\Projet;

class ProjetSeeder extends Seeder
{
    public function run(): void
    {
        Projet::create([
            'nom' => 'Site E-commerce',
            'description' => 'Boutique en ligne',
            'statut' => 'en_cours',
            'user_id' => 1,
        ]);
```

DatabaseSeeder Principal

Centraliser l'exécution des seeders

Le fichier `DatabaseSeeder.php` appelle tous les autres seeders :

```php
// database/seeders/DatabaseSeeder.php
class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        // Appeler les seeders dans l'ordre
        $this->call([
            UserSeeder::class,
            ProjetSeeder::class,
            TacheSeeder::class,
        ]);

        // Ou créer directement des données
        User::factory(10)->create();
    }
}
```

Commandes

Commande	Description
`php artisan db:seed`	Exécuter DatabaseSeeder
`php artisan migrate:fresh --seed`	Recréer les tables + seeder

5. Les Factories

Générer des données aléatoires

Définition

Une **factory** génère des données fictives réalistes grâce à la bibliothèque Faker. Idéal pour créer des données de test en masse.

Créer une factory

```
$ php artisan make:factory ProjetFactory
```

PHP

```php
// database/factories/ProjetFactory.php
class ProjetFactory extends Factory
{
    public function definition(): array
    {
        return [
            'nom' => fake()->sentence(3),
            'description' => fake()->paragraph(),
            'statut' => fake()->randomElement([
                'en_attente', 'en_cours', 'termine'
            ]),
            'date_debut' => fake()->dateTimeBetween('-1 month', 'now'),
            'user_id' => User::factory(),
        ];
    }
}
```

Méthodes Faker Utiles

Générer des données réalistes

Méthode	Résultat exemple
`fake()->name()`	Ahmed Ben Ali
`fake()->email()`	ahmed@example.com
`fake()->sentence(5)`	Lorem ipsum dolor sit amet.
`fake()->paragraph()`	Un paragraphe complet...
`fake()->numberBetween(1, 100)`	42
`fake()->date()`	2024-05-15
`fake()->boolean()`	true ou false
`fake()->randomElement([...])`	Un élément du tableau
`fake()->url()`	https://example.com
`fake()->imageUrl()`	URL d'une image placeholder

Localisation

Configurez la langue dans `config/app.php` : `'faker_locale' => 'fr_FR'`

6. Schéma Base de Données TaskFlow

Structure complète du projet

Diagramme des Tables

users
- id
- name
- email
- password
- timestamps

1 → N

projets
- id
- nom
- description
- statut
- user_id (FK)
- timestamps

1 → N

taches
- id
- titre
- description
- statut, priorite
- projet_id (FK)
- user_id (FK)
- timestamps

Relations

- Un **User** a plusieurs **Projets** (1:N)
- Un **Projet** a plusieurs **Tâches** (1:N)
- Une **Tâche** est assignée à un **User** (N:1)

Migration : Table Tâches

Exemple complet avec relations

PHP

```php
Schema::create('taches', function (Blueprint $table) {
    $table->id();

    // Informations de la tâche
    $table->string('titre');
    $table->text('description')->nullable();

    // Statut et priorité
    $table->enum('statut', ['todo', 'in_progress', 'done'])
          ->default('todo');
    $table->enum('priorite', ['basse', 'moyenne', 'haute'])
          ->default('moyenne');

    // Date d'échéance
    $table->date('date_echeance')->nullable();

    // Relations
    $table->foreignId('projet_id')
          ->constrained()
          ->onDelete('cascade');

    $table->foreignId('user_id')
          ->nullable()
          ->constrained()
          ->onDelete('set null');

    $table->timestamps();
});
```

Bonnes Pratiques

Conseils pour bien gérer vos migrations

✅ À Faire

- Une migration = une modification
- Noms explicites pour les tables
- Toujours définir la méthode `down()`
- Utiliser les clés étrangères
- Créer des index sur les colonnes recherchées
- Versionner les migrations avec Git

❌ À Éviter

- Modifier une migration déjà exécutée en prod
- Supprimer des migrations exécutées
- Oublier les contraintes de clés étrangères
- Mettre des données dans les migrations
- Ignorer l'ordre des migrations

> **Astuce de développement**
>
> Pendant le développement, utilisez `php artisan migrate:fresh --seed` pour repartir de zéro. En production, créez une nouvelle migration pour

Résumé du Chapitre

Migrations

Versionnement de la structure BDD

Seeders

Insertion de données initiales

Factories

Génération de données de test

Faker

Données aléatoires réalistes

Commandes essentielles

Commande	Action
`php artisan make:migration`	Créer une migration
`php artisan migrate`	Exécuter les migrations
`php artisan migrate:fresh --seed`	Reset complet + données
`php artisan make:seeder`	Créer un seeder
`php artisan make:factory`	Créer une factory

Prochain chapitre

Chapitre 5 : Eloquent ORM - Manipuler les données avec des modèles

Questions ?

Avez-vous des questions sur les migrations et les seeders ?

Prochaine séance : TP 4

Création de la base de données TaskFlow